

OS Banana - 2 Assignment

Name: Akshit Singhal

SRN: PES1UG24AM027

Q1)

A

```
[krish@archlinux PES1UG24AM027-pes-vcs]$ make test_objects
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
[krish@archlinux PES1UG24AM027-pes-vcs]$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check
```

B

```
All Phase 1 tests passed.
[krish@archlinux PES1UG24AM027-pes-vcs]$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
[krish@archlinux PES1UG24AM027-pes-vcs]$ █
```

Q2)

A

```
[krish@archlinux PEs1UG24AM027-pes-vcs]$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
[krish@archlinux PEs1UG24AM027-pes-vcs]$
```

B

```
[krish@archlinux PEs1UG24AM027-pes-vcs]$ find .pes/objects -type f
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
[krish@archlinux PEs1UG24AM027-pes-vcs]$ xxd .pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
00000000: 626c 6f62 2036 0068 656c 6c6f 0a          blob 6.hello.
[krish@archlinux PEs1UG24AM027-pes-vcs]$
```

Q3)

```
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
cat .pes/index    # Human-readable text format
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
Initialized empty PES repository in .pes/
Staged changes:
  staged:    file1.txt
  staged:    file2.txt

Unstaged changes:
(nothing to show)

Untracked files:
  untracked: .gitignore
  untracked: tree.h
  untracked: tree.c
  untracked: test_tree.c
  untracked: test_sequence.sh
  untracked: test_objects.c
  untracked: README.md
  untracked: pes.h
  untracked: pes.c
  untracked: object.h
  untracked: object.c
  untracked: Makefile
  untracked: index.h
  untracked: index.c
  untracked: commit.h
  untracked: commit.c

100644 2cf8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4 1776748274 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776748274 6 file2.txt
[krish@archlinux PES1UG24AM027-pes-vcs]$
```

Q4)

A

```
./pes log
Initialized empty PES repository in .pes/
[6366c9cf2b0a5620c1b0298873d5aaa362bcc147d27172771f1ab146734b9851] Initial commit
Committed: 6366c9cf2b0a... Initial commit
[ca47fbd39f73afb0ca275746ca9ed8011c2942bb741cd92762e5b9717cbdda52] Add world
Committed: ca47fbd39f73... Add world
[48539e8b3b3ee326825807cc16e5bbda79be782f6fe2f6f1a3c7e04634d6cc4f] Add farewell
Committed: 48539e8b3b3e... Add farewell
commit 48539e8b3b3ee326825807cc16e5bbda79be782f6fe2f6f1a3c7e04634d6cc4f
Author: Your Name <PES1UG24AM027>
Date: 1776748860

    Add farewell

commit ca47fbd39f73afb0ca275746ca9ed8011c2942bb741cd92762e5b9717cbdda52
Author: Your Name <PES1UG24AM027>
Date: 1776748860

    Add world

commit 6366c9cf2b0a5620c1b0298873d5aaa362bcc147d27172771f1ab146734b9851
Author: Your Name <PES1UG24AM027>
Date: 1776748860

    Initial commit

[krish@archlinux PES1UG24AM027-pes-vcs]$
```

B

```
[krish@archlinux PES1UG24AM027-pes-vcs]$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/2c/f8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4
.pes/objects/48/539e8b3b3ee326825807cc16e5bbda79be782f6fe2f6f1a3c7e04634d6cc4f
.pes/objects/63/66c9cf2b0a5620c1b0298873d5aaa362bcc147d27172771f1ab146734b9851
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/85/e367b76c2417b47a95f46208835ad8700dd58335171c94a497974be6191900
.pes/objects/96/c0d5c28dd3b719606c36723cd16aefbc73b3dc8b74de61070183357d6968b9
.pes/objects/ca/47fbd39f73afb0ca275746ca9ed8011c2942bb741cd92762e5b9717cbdda52
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/f1/0239b620372163c2dd06ac33189180c55edf1c56b15990a1cf44e00b1b0e59
.pes/refs/heads/main
[krish@archlinux PES1UG24AM027-pes-vcs]$
```

C

```
[krish@archlinux PEs1UG24AM027-pes-vcs]$ cat .pes/refs/heads/main  
48539e8b3b3ee326825807cc16e5bbda79be782f6fe2f6f1a3c7e04634d6cc4f  
[krish@archlinux PEs1UG24AM027-pes-vcs]$ cat .pes/HEAD  
ref: refs/heads/main  
[krish@archlinux PEs1UG24AM027-pes-vcs]$ █
```

Q5.1

Checkout updates `.pes/HEAD` to point to the target branch, reads the commit hash from `.pes/refs/heads/<branch>`, loads its tree, and rewrites the working directory to match that tree. It also updates the index to reflect the new snapshot. The complexity comes from safely handling file overwrites, deletions, and directory structure changes while avoiding loss of uncommitted work.

Q5.2

For each tracked file, compare the working directory state with the index by recomputing the hash. If a file is modified relative to the index and also differs between the current tree and target tree, checkout must refuse. This detects conflicts by comparing working directory vs index and index vs target tree.

Q5.3

In detached HEAD, commits are created normally but no branch points to them, so they become unreachable once you move away. The user can recover them if they know the commit hash by creating a new branch pointing to it, or by checking it out again before it is garbage collected.

Q6.1

Start from all branch heads and traverse commits, trees, and blobs using DFS or BFS, marking all reachable object hashes in a hash set. Then scan the object store and delete any object not in the reachable set. With 100k commits and shared history, traversal is roughly 100k commits plus their trees and blobs, not multiplied by branches.

Q6.2

If GC runs while a commit is being created, it may delete objects (like blobs or trees) that are written but not yet referenced by any commit. The commit then ends up pointing to missing objects. Git avoids this by writing objects before updating refs, delaying deletion of recent objects, and using safeguards like timestamps and locks to prevent such races